Subsystem 5 Forum Database Manual (MySQL)

Oct 22, 2026 · @Michael Pieterse

1. Overview

The Subsystem 5 forum database is 11 MySQL tables, 3 views, and 12 indexes in one database called `mason_forum`. It stores everything the Community Forum needs: posts, threaded replies, tags, upvotes, flags, and moderation history. This manual covers every piece of it, how to run it, how to test it, and what our Java code has to do on top of it.

The two files

File	What it does	When to run it
`subsystem5_forum_mysql.sql`	Drops and rebuilds the whole schema, then seeds the 6 forum categories	First setup, and any time you want a clean database
`subsystem5_forum_tests.sql`	Loads sample data, runs the read queries our API needs, and tries 11 inserts that should fail	After the schema script, to prove the schema works

The schema script is safe to re-run. It drops everything first. That also means it wipes any data you added, so don't run it against anything you care about.

Version requirements

- **MySQL 8.0.16 or newer.** Older versions read `CHECK` constraints but never enforce them. Your inserts will succeed when they should fail, and nothing will warn you.
- **MySQL 8.0.13 or newer** for `DEFAULT (UUID())` on the ID columns. Anything new enough for the first rule covers this one.
- **InnoDB storage engine.** It's the default, and every table sets it explicitly. MyISAM ignores foreign keys entirely.

Check your version with `SELECT VERSION();`.

The design rule: structure in the database, behavior in Java

The database enforces **shape**. Every post has a real author. Every vote is unique. A reply's parent is in the same thread. A flag can only be resolved once.

The database does **not** enforce **behavior**. There are no triggers and no stored procedures. Updating reply counts, blocking self-votes, closing a flag after moderation, and soft-deleting removed content all happen in our Java service layer. Section 8 lists every one of those rules with the SQL pattern to use.

Why split it this way? Our business logic lives in one place, in a language the whole team reads, and it's covered by our unit tests. The downside is real, though. Anyone who writes to these tables directly (a teammate in Workbench, another subsystem skipping our API) can leave the counters wrong. Section 10 includes a drift check that catches this.

2. Setup

Setup takes about five minutes: connect, run one script, check four things.

Run the schema script

1. Open MySQL Workbench and connect to your local MySQL server (usually `localhost:3306`, user `root`).
2. Run `SELECT VERSION();` and confirm it's 8.0.16 or higher.
3. Go to **File > Open SQL Script** and pick `subsystem5_forum_mysql.sql`.
4. Click the **plain lightning bolt** (Execute All). Don't use the lightning bolt with the cursor on it. That one only runs the statement your cursor is sitting in.
5. Watch the Output panel at the bottom. Every row should have a green check. The last one is the `INSERT INTO category` with 6 rows affected.
6. Right-click the **Schemas** panel on the left and hit **Refresh All**. You should see `mason_forum` with 11 tables and 3 views.

Verify it worked

Run these four queries. If all four match, the schema is good.

```
USE mason_forum;

-- 11 rows expected
SELECT table_name FROM information_schema.tables
 WHERE table_schema = 'mason_forum' AND table_type = 'BASE TABLE';

-- 3 rows expected
SELECT table_name FROM information_schema.views
 WHERE table_schema = 'mason_forum';

-- 6 rows expected
SELECT id, name, sort_order FROM category ORDER BY sort_order;

-- should list ck_ constraints; if empty, your MySQL is too old
SELECT constraint_name, check_clause FROM
information_schema.check_constraints
 WHERE constraint_schema = 'mason_forum';
```

Other ways to run it

- **Command line:** `mysql -u root -p < subsystem5_forum_mysql.sql`
- **IntelliJ / DBeaver:** open the file, pick your MySQL connection, and run the whole script (not just the selected statement).
- **Docker:** if Shared Core gives us a MySQL container, mount the script into `/docker-entrypoint-initdb.d/`. MySQL runs anything in that folder the first time the container starts.

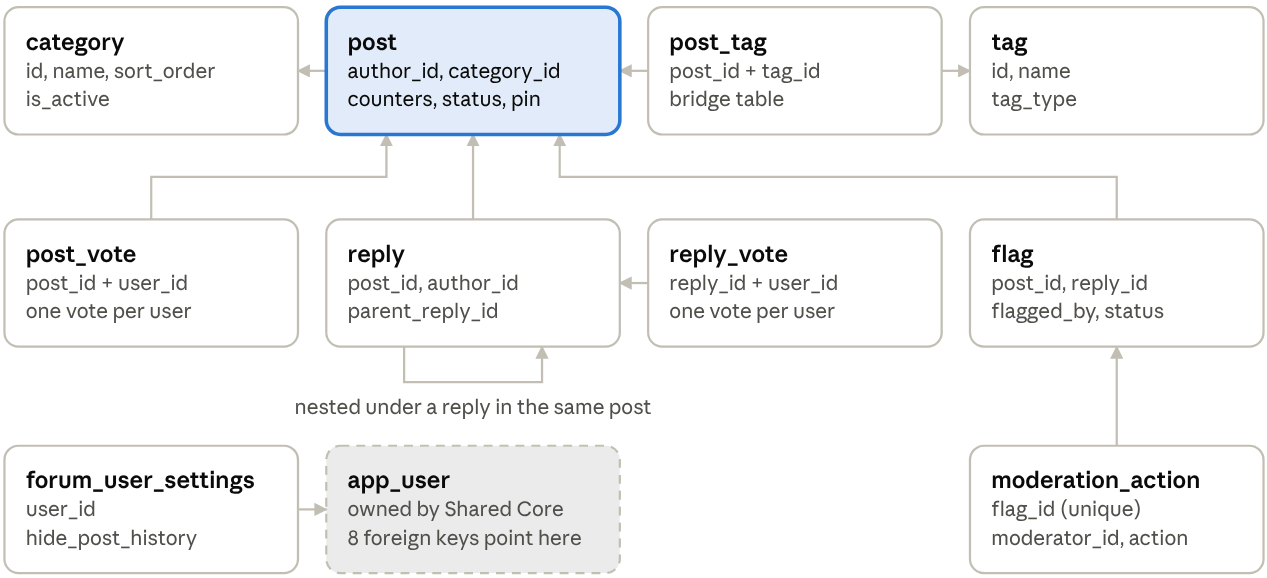
Resetting

Run the schema script again. It drops the views and all 11 tables, then rebuilds them. Any sample data from the test script is gone afterward, so re-run that too if you need it.

3. Schema map

`post` is the center of the schema: six of the other ten tables point at it directly or through `reply`.

Posts sit at the center; replies, votes, tags and flags hang off them



Arrows point from child to parent. Dashed box: Shared Core's table.
Every column naming a person (author_id, user_id, flagged_by, moderator_id) also points at app_user.

forum schema · 11 tables

The user lines are left off the drawing to keep it readable. Section 5 lists all 19 foreign keys.

4. Table-by-table reference

Every column in all 11 tables, in the order the script creates them. "Java" in the Rules column means the database won't stop a bad value there; our service code has to.

Two patterns repeat across tables:

- **UUID keys.** `CHAR(36) DEFAULT (UUID())`. Leave `id` out of an insert and MySQL generates one. In Java, generate it yourself with `UUID.randomUUID().toString()` so you know the ID without a second query.
- **Lookup-table keys.** `category` and `tag` use `INT AUTO_INCREMENT`. They're small, admin-managed lists, so a readable number is fine.

app_user (owned by Shared Core)

A stand-in so our foreign keys have something to point at. In the integrated build, Shared Core owns the real user table and we stop creating this one (see section 12).

Column	Type	Default	Rules
id	CHAR(36)	UUID()	Primary key
name	VARCHAR(120)	none	Required
gmu_email	VARCHAR(254)	none	Required, unique
role	VARCHAR(10)	none	Must be STUDENT, MENTOR, STAFF, or ADMIN

category

The fixed list of forum sections. Admins manage it; users can post in any category but can't create one. The schema script seeds the six from the SOW.

Column	Type	Default	Rules
id	INT	AUTO_INCREMENT	Primary key
name	VARCHAR(60)	none	Required, unique
description	VARCHAR(400)	NULL	Optional
sort_order	SMALLINT	0	Display order, lowest first
is_active	TINYINT(1)	1	0 or 1. Set to 0 to hide a category without deleting it

You can't delete a category that has posts. The foreign key blocks it. Deactivate it instead.

post

A top-level thread. Three columns are **denormalized counters**: copies of numbers you could compute, stored so the list pages don't have to count rows every time.

Column	Type	Default	Rules
id	CHAR(36)	UUID()	Primary key
author_id	CHAR(36)	none	Required, must exist in app_user
category_id	INT	none	Required, must exist in category

Column	Type	Default	Rules
title	VARCHAR(200)	none	Required
body	TEXT	none	Required, up to 64 KB
vote_score	INT	0	Counter. Must be 0 or more. Java keeps it equal to the post's vote count
reply_count	INT	0	Counter. Must be 0 or more. Java keeps it equal to the post's reply count
helpful_reply_id	CHAR(36)	NULL	Must be a reply on this same post. Java checks only the post author sets it
is_pinned	TINYINT(1)	0	0 or 1
pinned_by	CHAR(36)	NULL	Must exist in app_user. Required when pinned, empty when not. Java checks the pinner is STAFF
status	VARCHAR(10)	'visible'	visible or removed
created_at	DATETIME	now	Set once
last_activity_at	DATETIME	now	Java bumps it on every new reply. Drives Trending

reply

A response inside a thread. `parent_reply_id` is empty for a direct reply to the post, or points at another reply for a nested one.

Column	Type	Default	Rules
id	CHAR(36)	UUID()	Primary key
post_id	CHAR(36)	none	Required, must exist in post
parent_reply_id	CHAR(36)	NULL	If set, must be a reply on the same post. Java checks it isn't the reply's own id
author_id	CHAR(36)	none	Required, must exist in app_user
body	TEXT	none	Required

Column	Type	Default	Rules
vote_score	INT	0	Counter, 0 or more, kept by Java
status	VARCHAR(10)	'visible'	visible or removed
created_at	DATETIME	now	Set once

There's also a `UNIQUE (id, post_id)` pair. It looks redundant since `id` is already unique. It's there so other tables can point at a reply **and** its post together, which is how the database proves a reply belongs to a given thread.

tag

Optional labels, like an employer or a role name.

Column	Type	Default	Rules
id	INT	AUTO_INCREMENT	Primary key
name	VARCHAR(80)	none	Required, unique
tag_type	VARCHAR(10)	'other'	employer, role, or other

post_tag

Connects posts and tags. A post can have many tags; a tag can be on many posts.

Column	Type	Default	Rules
post_id	CHAR(36)	none	Part of the primary key. Deleted with its post
tag_id	INT	none	Part of the primary key. Deleted with its tag

post_vote and reply_vote

One row per upvote. The two tables are identical except for what they point at.

Column	Type	Default	Rules
post_id / reply_id	CHAR(36)	none	Part of the primary key. Deleted with its post or reply
user_id	CHAR(36)	none	Part of the primary key. Must exist in app_user
vote_value	TINYINT	1	Must be exactly 1. The SOW only has upvotes
created_at	DATETIME	now	Set once

The two-column primary key is what makes "one vote per user" automatic. A second vote from the same user is a duplicate key, so MySQL rejects it. Java still has to block people voting on their own content.

flag

A report from a user. `reply_id` empty means the whole post is flagged; set means one reply inside it.

Column	Type	Default	Rules
id	CHAR(36)	UUID()	Primary key
post_id	CHAR(36)	none	Required. Deleted with its post
reply_id	CHAR(36)	NULL	If set, the reply must be on post_id
flagged_by	CHAR(36)	none	Must exist in app_user
reason	VARCHAR(500)	none	Required
status	VARCHAR(10)	'open'	open, actioned, or dismissed. Java changes it when a moderator acts
created_at	DATETIME	now	Set once

moderation_action

The audit log. One row per resolved flag, never more.

Column	Type	Default	Rules
id	CHAR(36)	UUID()	Primary key
flag_id	CHAR(36)	none	Required, unique, must exist in flag
moderator_id	CHAR(36)	none	Must exist in app_user. Java checks they're STAFF or ADMIN
action	VARCHAR(10)	none	remove, warn, or dismiss
reason	VARCHAR(500)	none	Required. The SOW asks for a reason on every action
created_at	DATETIME	now	Set once

forum_user_settings

Forum-only settings for a user. A user with no row here gets the defaults, so you only insert when someone changes a setting.

Column	Type	Default	Rules
user_id	CHAR(36)	none	Primary key. Deleted with its user
hide_post_history	TINYINT(1)	0	0 or 1. The SOW's opt-out for posting history
reputation	INT	0	Java decides how it's earned

5. Constraint catalog

The schema has 19 foreign keys, 14 checks, and 7 unique rules. Every one has a name, so when MySQL rejects something, the error message tells you exactly which rule fired. Search this section for that name.

Foreign keys

"Cascade" means deleting the parent row deletes this row too. "Block" means MySQL refuses to delete the parent while this row exists (error 1451).

Name	From	To	When the parent is deleted
fk_post_author	post.author_id	app_user.id	Block
fk_post_category	post.category_id	category.id	Block
fk_post_pinned_by	post.pinned_by	app_user.id	Block
fk_post_helpful_reply	post (helpful_reply_id, id)	reply (id, post_id)	Block
fk_reply_post	reply.post_id	post.id	Cascade
fk_reply_author	reply.author_id	app_user.id	Block
fk_reply_parent	reply (parent_reply_id, post_id)	reply (id, post_id)	Cascade
fk_post_tag_post	post_tag.post_id	post.id	Cascade
fk_post_tag_tag	post_tag.tag_id	tag.id	Cascade
fk_post_vote_post	post_vote.post_id	post.id	Cascade
fk_post_vote_user	post_vote.user_id	app_user.id	Block
fk_reply_vote_reply	reply_vote.reply_id	reply.id	Cascade
fk_reply_vote_user	reply_vote.user_id	app_user.id	Block
fk_flag_post	flag.post_id	post.id	Cascade
fk_flag_reply	flag (reply_id, post_id)	reply (id, post_id)	Cascade
fk_flag_user	flag.flagged_by	app_user.id	Block
fk_mod_flag	moderation_action.flag_id	flag.id	Block
fk_mod_user	moderation_action.moderator_id	app_user.id	Block
fk_fus_user	forum_user_settings.user_id	app_user.id	Cascade

The composite keys, and why they matter

Three foreign keys point at a **pair** of columns, `reply (id, post_id)`, instead of just `reply.id`. That's what stops cross-thread mistakes.

Let's say a bug in our code tries to nest a reply from post A under a reply that lives in post B. With a plain key on `parent_reply_id`, MySQL would accept it, because the parent reply exists. With the pair, MySQL looks for a reply with that id **and** post A's id. It doesn't find one, so the insert fails.

The same trick guards the helpful reply (must be on this post) and reply flags (the flagged reply must be on the flagged post).

When the first column of the pair is NULL, MySQL skips the check completely. That's how "no parent", "no helpful reply yet", and "flag on the whole post" stay legal.

What deletes actually do

The cascades are set up so deleting content cleans up after itself, and the blocks protect two things: users, and the moderation log.

- **Delete a post:** its replies, tags, votes, and flags go with it. Clear `helpful_reply_id` first, or the post's own pointer to its reply can block the delete.
- **Delete a reply:** its nested replies, their votes, and their flags go too, all the way down.
- **Delete anything with a resolved flag:** blocked. The cascade reaches the flag, but `fk_mod_flag` won't let a flag with a moderation record disappear. This is deliberate. The SOW says moderation decisions are logged and auditable, so removed content gets `status = 'removed'`, never `DELETE`.
- **Delete a user:** blocked if they've posted, replied, voted, flagged, pinned, or moderated anything. In practice, users are deactivated by Shared Core, not deleted.
- **Delete a category:** blocked while it has posts. Set `is_active = 0` instead.

The short version: our app should almost never run `DELETE` on forum content. Soft delete with `status` instead.

Checks

Name	Table	Rule
ck_app_user_role	app_user	role IN ('STUDENT','MENTOR','STAFF','ADMIN')

Name	Table	Rule
ck_category_active	category	is_active IN (0,1)
ck_post_pinned	post	is_pinned IN (0,1)
ck_post_status	post	status IN ('visible','removed')
ck_post_pin_pair	post	pinned has a pinner; unpinned has none
ck_post_counts	post	vote_score >= 0 AND reply_count >= 0
ck_reply_score	reply	vote_score >= 0
ck_reply_status	reply	status IN ('visible','removed')
ck_tag_type	tag	tag_type IN ('employer','role','other')
ck_post_vote_value	post_vote	vote_value = 1
ck_reply_vote_value	reply_vote	vote_value = 1
ck_flag_status	flag	status IN ('open','actioned','dismissed')
ck_mod_action	moderation_action	action IN ('remove','warn','dismiss')
ck_fus_hide	forum_user_settings	hide_post_history IN (0,1)

The counter checks are a safety net, not the main defense. If Java ever subtracts more votes than exist, the `UPDATE` fails with error 3819 instead of quietly saving a negative score.

Unique rules

Name	Table	Columns	What it prevents
uq_app_user_email	app_user	gmu_email	Two accounts on one email
uq_category_name	category	name	Duplicate categories
uq_tag_name	tag	name	"Capital One" twice
uq_reply_id_post	reply	id, post_id	Nothing by itself. It exists so the composite foreign keys have a target
uq_mod_flag	moderation_action	flag_id	Resolving one flag twice

Name	Table	Columns	What it prevents
pk_post_vote	post_vote	post_id, user_id	Double voting on a post
pk_reply_vote	reply_vote	reply_id, user_id	Double voting on a reply

The one check we couldn't keep

The Oracle version had `ck_reply_not_own_parent`, which stopped a reply from being its own parent. MySQL won't allow a check on a column that also has `ON DELETE CASCADE` (error 3823), and `parent_reply_id` needs the cascade. So that rule moved to Java. Section 8 covers it.

6. Indexes and full-text search

The script adds 11 regular indexes and 1 full-text index, each matched to a specific query the API runs. InnoDB also creates an index on every foreign key automatically if one doesn't already exist, so joins on keys are covered without these.

Index	Columns	The query it speeds up
ix_post_category_activity	post (category_id, status, last_activity_at)	Browsing one category, newest activity first
ix_post_author	post (author_id)	A user's posting history
ix_post_unanswered	post (status, reply_count, created_at)	The Unanswered tab
ix_reply_post	reply (post_id, created_at)	Loading a thread's replies in order
ix_reply_author	reply (author_id)	A user's reply history
ix_reply_parent	reply (parent_reply_id, post_id)	Finding nested replies under a parent
ix_post_tag_tag	post_tag (tag_id)	"All posts tagged Capital One"

Index	Columns	The query it speeds up
ix_post_vote_user	post_vote (user_id)	"Has this user voted on these posts?"
ix_reply_vote_user	reply_vote (user_id)	Same, for replies
ix_flag_status	flag (status, created_at)	The moderation queue, oldest open flag first
ix_flag_post_reply	flag (reply_id, post_id)	Finding flags on a given reply
ftx_post_title_body	post (title, body), FULLTEXT	Forum search

How column order works

A multi-column index only helps when the query filters on its columns **left to right**. `ix_post_category_activity` helps `WHERE category_id = 3 AND status = 'visible' ORDER BY last_activity_at`. It does nothing for `WHERE status = 'visible'` alone, because `category_id` comes first. If you add a query that sorts or filters differently, check it with `EXPLAIN` (section 11).

Full-text search

The SOW asks for full-text search across post titles and bodies. MySQL handles that with `MATCH ... AGAINST`:

```
SELECT id, title,
       MATCH(title, body) AGAINST ('capital one power day') AS relevance
  FROM post
 WHERE MATCH(title, body) AGAINST ('capital one power day')
   AND status = 'visible'
ORDER BY relevance DESC
LIMIT 20 OFFSET 0;
```

Things that will trip you up:

- **The column list must match the index exactly.** `MATCH(title, body)` works. `MATCH(title)` alone fails with error 1191, because there's no full-text index on just `title`.
- **Short words are ignored.** InnoDB skips words under 3 characters, so "QA" and "PM" won't match anything. Common words like "the" and "about" are skipped too.

- **Natural language mode** (the default above) ranks by relevance. For exact control, use boolean mode: `AGAINST ('+capital -onsite' IN BOOLEAN MODE)` means must contain "capital", must not contain "onsite".
- **Replies aren't indexed.** Search covers posts only. If the team wants reply text searchable later, add a FULLTEXT index on `reply (body)` and run a second query.

If full-text gives weird results during a demo, `title LIKE CONCAT('%', ?, '%')` is a fine fallback for small data. It's slow on big tables, but we won't have big tables.

7. Views

The three views are saved queries for the three special list pages in the SOW. You query them like tables. They hold no data of their own, so they're always current.

View	Shows	Columns
v_unanswered_posts	Visible posts with zero replies	id, title, category_id, category_name, author_id, vote_score, created_at
v_trending_posts	Visible posts active in the last 7 days, with a score	id, title, category_id, vote_score, reply_count, last_activity_at, trend_score
v_moderation_queue	Every open flag, with the flagged post's title	flag_id, post_id, post_title, reply_id, flagged_by, reason, created_at

The views don't sort or paginate. That's on purpose, so each endpoint can pick its own order. Add `ORDER BY` and `LIMIT` when you query them:

```
-- Unanswered tab: oldest first, so long-waiting posts get seen
SELECT * FROM v_unanswered_posts ORDER BY created_at ASC LIMIT 20 OFFSET 0;

-- Trending tab
SELECT * FROM v_trending_posts ORDER BY trend_score DESC, last_activity_at
DESC LIMIT 20;

-- Moderation queue: oldest flag first
SELECT * FROM v_moderation_queue ORDER BY created_at ASC LIMIT 50;
```

How the trending score works

`trend_score = reply_count × 2 + vote_score`. A reply counts double because it takes more effort than a click. The 7-day window keeps old threads from sitting on top forever. Both numbers are guesses, not research. If the team wants to tune them, edit the view in the schema script and re-run it.

Two things the views depend on

- **Accurate counters.** Unanswered and Trending read `reply_count` and `vote_score` straight off the post row. If Java forgets to update them, these pages are wrong. That's the biggest practical risk in this design.
- **The API checking roles.** `v_moderation_queue` has no idea who's asking. The endpoint must confirm the caller is STAFF or ADMIN through Shared Core before running it, or students can see every flag.

8. Rules the Java layer must enforce

These are the rules the database can't or won't enforce. If our service code skips one, nothing stops bad data from landing. Each rule below has the SQL to run, in order.

Use a transaction whenever two tables change together

Adding a reply means inserting a reply **and** bumping the post's counter. If the second statement fails and the first already committed, the counter is wrong forever. Wrap them:

```
try (Connection conn = dataSource.getConnection()) {
    conn.setAutoCommit(false);
    try (PreparedStatement ins = conn.prepareStatement(
              "INSERT INTO reply (id, post_id, parent_reply_id, author_id,
body) VALUES (?, ?, ?, ?, ?)");
         PreparedStatement upd = conn.prepareStatement(
              "UPDATE post SET reply_count = reply_count + 1, last_activity_at
= NOW() WHERE id = ?")) {
        String replyId = UUID.randomUUID().toString();
        ins.setString(1, replyId);
        ins.setString(2, postId);
        ins.setString(3, parentReplyId);   // null for a top-level reply
        ins.setString(4, authorId);
        ins.setString(5, body);
        ins.executeUpdate();
```

```
            upd.setString(1, postId);
            upd.executeUpdate();
            conn.commit();
        } catch (SQLException e) {
            conn.rollback();
            throw e;
        }
    }
```

Always use `PreparedStatement` with `?` placeholders. Never build SQL by gluing strings together. Forum posts are user-typed text, and string-built SQL is how SQL injection happens.

The rules

#	Rule	When	What Java does
1	Keep reply_count and last_activity_at right	New reply	Insert reply, then `reply_count + 1` and `last_activity_at = NOW()`, one transaction
2	A reply isn't its own parent	New reply	Reject if `parent_reply_id` equals the new id. Also confirm the parent's status is visible
3	No voting on your own content	New vote	Look up the author. Reject with 403 if it's the voter
4	Keep vote_score right	Vote or unvote	Insert vote + `vote_score + 1`, or delete vote + `vote_score - 1`, one transaction
5	Only the post author marks helpful	Mark helpful	Compare caller to `post.author_id`. The database already checks the reply is on this post
6	Only staff pin	Pin	Check role is STAFF through Shared Core. Set `is_pinned = 1` and `pinned_by` together
7	Only staff and admins moderate	Any moderation call	Check role is STAFF or ADMIN through Shared Core

#	Rule	When	What Java does
8	Only open flags get acted on	Moderation action	See the moderation flow below
9	Removed content stays hidden	Every read	Add `status = 'visible'` to every public query
10	Respect hidden history	Profile history	Skip the user's posts when `hide_post_history = 1`, unless the caller is that user or staff
11	Never hard-delete moderated content	Removal	Set `status = 'removed'`. Never `DELETE`

Voting

```
-- 1. Who wrote it?
SELECT author_id FROM post WHERE id = ? AND status = 'visible';
-- if author_id == caller: return 403, stop

-- 2. In one transaction:
INSERT INTO post_vote (post_id, user_id) VALUES (?, ?);
UPDATE post SET vote_score = vote_score + 1 WHERE id = ?;
```

If step 2's insert throws error 1062, the user already voted. Roll back and return 409. You don't need to check first; the primary key does it for you, and it's race-safe. Two clicks arriving at the same millisecond still produce one vote.

Removing a vote is the reverse: `DELETE FROM post_vote WHERE post_id = ? AND user_id = ?`, and only run `vote_score - 1` if the delete actually removed a row (`executeUpdate()` returned 1). Replies work the same way with `reply_vote` and `reply`.

The moderation flow

The SOW describes flag, then review, then action. Here is the action step, all in one transaction:

```
-- 1. Lock the flag so two moderators can't act at once
SELECT status, post_id, reply_id FROM flag WHERE id = ? FOR UPDATE;
-- if status != 'open': roll back, return 409 "already resolved"

-- 2. Log it
INSERT INTO moderation_action (flag_id, moderator_id, action, reason)
VALUES (?, ?, ?, ?);

-- 3. Close the flag
UPDATE flag SET status = ? WHERE id = ?;   -- 'dismissed' for dismiss,
'actioned' otherwise

-- 4. Only for action = 'remove'
UPDATE reply SET status = 'removed' WHERE id = ?;   -- when the flag has a
reply_id
UPDATE post  SET status = 'removed' WHERE id = ?;   -- when it doesn't
```

If two moderators still race, `uq_mod_flag` makes the second insert fail with 1062. Treat it the same as "already resolved". After commit, call Notifications' `POST /notifications/dispatch` with the moderation template.

Hiding posting history

Users with no settings row get the defaults. This one statement creates the row or updates it:

```
INSERT INTO forum_user_settings (user_id, hide_post_history) VALUES (?, ?)
ON DUPLICATE KEY UPDATE hide_post_history = VALUES(hide_post_history);
```

Turning MySQL errors into HTTP responses

The SOW says every error uses Shared Core's `{"error", "code", "detail"}` format. Map these MySQL codes (`SQLException.getErrorCode()`):

MySQL code	Meaning	HTTP	Example
1062	Duplicate key	409 Conflict	Second vote, second moderation action

MySQL code	Meaning	HTTP	Example
1452	Foreign key target missing	400 or 404	Reply on a post that doesn't exist
1451	Row is still referenced	409 Conflict	Deleting a category with posts
3819	Check constraint failed	400 Bad Request	Bad role, bad status, negative counter
1406	Value too long for column	400 Bad Request	Title over 200 characters
1048	Required column is NULL	400 Bad Request	Post with no title

These are the backstop. Per the SOW, validate input before it reaches the database and return specific field errors. The database errors should only fire when our validation missed something.

Open questions for the team

- Does `reply_count` include removed replies? Right now it counts every reply row, and the drift check in section 10 assumes that. If we want it to count only visible replies, rule 11 needs a `reply_count - 1` and the drift check needs a status filter.
- Should one user be able to flag the same post twice? Nothing blocks it now. A `UNIQUE (post_id, reply_id, flagged_by)` would.
- How is `reputation` earned? The column exists, but nothing updates it yet.

9. Cookbook: SQL for each endpoint

One recipe per endpoint our subsystem exposes in the SOW. Every `?` is a `PreparedStatement` parameter. Copy, adjust, and plug in.

Pagination (every list endpoint)

The SOW requires page number and page size, a max size of 50, and a total count in the response. Use two queries with the same `WHERE`:

```
-- page = 1-based page number, size = min(requested, 50)
-- OFFSET = (page - 1) * size
SELECT ... FROM post WHERE ... ORDER BY ... LIMIT ? OFFSET ?;
SELECT COUNT(*) FROM post WHERE ...;   -- goes in the response metadata
```

Cap `size` at 50 in Java before it reaches the query. MySQL won't do it for you.

GET /posts

Filters: category, tag, and sort by recency, votes, or unanswered. Pinned posts float to the top.

```
SELECT p.id, p.title, c.name AS category, u.name AS author,
       p.vote_score, p.reply_count, p.is_pinned, p.last_activity_at
  FROM post p
  JOIN category c ON c.id = p.category_id
  JOIN app_user u ON u.id = p.author_id
 WHERE p.status = 'visible'
   AND (? IS NULL OR p.category_id = ?)                  -- category filter,
or null
   AND (? IS NULL OR EXISTS (SELECT 1 FROM post_tag pt
                             JOIN tag t ON t.id = pt.tag_id
                            WHERE pt.post_id = p.id AND t.name = ?))   --
tag filter
 ORDER BY p.is_pinned DESC, p.last_activity_at DESC     -- sort=recent
 LIMIT ? OFFSET ?;
```

Swap the `ORDER BY` for the other sorts:

- **votes:** `ORDER BY p.is_pinned DESC, p.vote_score DESC, p.created_at DESC`
- **unanswered:** add `AND p.reply_count = 0` and use `ORDER BY p.created_at ASC`, or just query `v_unanswered_posts`

Pass the category id twice (once for the `IS NULL` test, once for the match). Same for the tag name. Never paste the sort direction in from user input. Map `"recent"` / `"votes"` / `"unanswered"` to one of three fixed strings in Java.

GET /posts/{id}

Two queries: the post, then its replies.

```
SELECT p.*, c.name AS category_name, u.name AS author_name
  FROM post p
  JOIN category c ON c.id = p.category_id
  JOIN app_user u ON u.id = p.author_id
 WHERE p.id = ? AND p.status = 'visible';     -- no row = 404

SELECT GROUP_CONCAT(t.name) AS tags
  FROM post_tag pt JOIN tag t ON t.id = pt.tag_id
 WHERE pt.post_id = ?;

SELECT r.id, r.parent_reply_id, r.author_id, u.name AS author_name,
       r.body, r.vote_score, r.created_at,
       (r.id = ?) AS is_helpful                -- pass the post's
helpful_reply_id
  FROM reply r
  JOIN app_user u ON u.id = r.author_id
 WHERE r.post_id = ? AND r.status = 'visible'
 ORDER BY r.created_at;
```

The replies come back flat. Build the tree in Java: put every reply in a map by id, then attach each one to its `parent_reply_id`. Replies with no parent are the top level. One pass, no recursive SQL needed.

To show which replies the current user already upvoted:

```
SELECT reply_id FROM reply_vote
 WHERE user_id = ? AND reply_id IN (SELECT id FROM reply WHERE post_id = ?);
```

POST /posts

One transaction. Tags are created on first use.

```
INSERT INTO post (id, author_id, category_id, title, body) VALUES (?, ?, ?,
?, ?);

-- for each tag name in the request:
INSERT IGNORE INTO tag (name, tag_type) VALUES (?, ?);   -- no-op if it
exists
INSERT INTO post_tag (post_id, tag_id)
SELECT ?, id FROM tag WHERE name = ?;
```

Check that the category exists **and** has `is_active = 1` before inserting. The foreign key only checks it exists.

POST /posts/{id}/replies

The transaction in section 8. Before it, confirm the post is visible, and if `parent_reply_id` was sent, that the parent is a visible reply on this post. After commit, notify the post author through Notifications.

POST /posts/{id}/flag

```
INSERT INTO flag (id, post_id, reply_id, flagged_by, reason) VALUES (?, ?, ?, ?, ?);
```

`reply_id` is null to flag the whole post. A 1452 error here means the reply isn't on that post: return 400. After commit, notify staff that the queue has a new item.

GET /moderation/queue

Staff and admin only, checked through Shared Core first.

```
SELECT * FROM v_moderation_queue ORDER BY created_at ASC LIMIT ? OFFSET ?;
SELECT COUNT(*) FROM v_moderation_queue;
```

User posting history

For someone else's profile. Pass the profile owner's id three times.

```
SELECT 'post' AS kind, p.id, p.title AS text, p.created_at
  FROM post p
 WHERE p.author_id = ? AND p.status = 'visible'
   AND NOT EXISTS (SELECT 1 FROM forum_user_settings s
                   WHERE s.user_id = ? AND s.hide_post_history = 1)
UNION ALL
SELECT 'reply', r.id, LEFT(r.body, 120), r.created_at
  FROM reply r
 WHERE r.author_id = ? AND r.status = 'visible'
   AND NOT EXISTS (SELECT 1 FROM forum_user_settings s
                   WHERE s.user_id = r.author_id AND s.hide_post_history =
1)
 ORDER BY created_at DESC
 LIMIT ? OFFSET ?;
```

Skip the `NOT EXISTS` lines when the viewer is the owner or staff.

10. Testing

`subsystem5_forum_tests.sql` proves the schema in about two minutes. Run the schema script first, then the four parts of the test script in order, **in the same Workbench tab**. The script saves IDs in `@variables`, and those only live as long as your connection.

Part A: sample data (Execute All)

Loads a small, realistic forum:

What	Rows	Details
Users	5	Alice and Bob (students), Carol (mentor), Sam (staff), Ada (admin)
Posts	2	p1: Capital One interview question by Alice. p2: negotiation question by Bob
Tags	2	"Capital One" and "Software Engineer Intern", both on p1
Replies	2	r1: Bob answers p1. r2: Carol replies under r1
Votes	4	Bob and Carol upvote p1. Alice and Carol upvote r1

What	Rows	Details
Flags	2	f1: Alice flags r2. f2: Carol flags p2
Moderation	2	Sam dismisses f1. Ada removes p2

Part A also runs the counter updates and moderation follow-ups by hand, exactly as section 8 says Java should. Read it as a worked example of those rules.

Part B: read queries (Execute All)

#	Query	Expected result
1	Row counts	app_user 5, category 6, post 2, reply 2, post_vote 2, flag 2, moderation_action 2
2	Post list	1 row: p1. p2 is removed
3	Thread detail for p1	2 replies. r1 has is_helpful = 1. r2's parent is r1
4	Tag filter "Capital One"	p1
5	Full-text "power day technical"	p1 with a relevance above 0
6a	v_unanswered_posts	Empty. p1 has replies, p2 is removed
6b	v_trending_posts	p1, trend_score 6 (2 replies × 2 + 2 votes)
6c	v_moderation_queue	Empty. Both flags are resolved
7	Moderation log	2 rows: Sam dismiss, Ada remove
8	Posting history	Only Alice's post. Bob hid his history
9	Counter drift checks	0 rows from both

Part C: negative tests (one at a time)

Each of these **should fail**. Put your cursor on the statement and press Ctrl+Enter. Execute All would stop at the first error and skip the rest.

#	Tries to	Expected error	Rule that catches it
C1	Vote twice	1062	pk_post_vote
C2	Vote with value 2	3819	ck_post_vote_value
C3	Create a HACKER user	3819	ck_app_user_role
C4	Pin with no pinner	3819	ck_post_pin_pair
C5	Nest a reply under another post's reply	1452	fk_reply_parent
C6	Mark another post's reply helpful	1452	fk_post_helpful_reply
C7	Flag a reply under the wrong post	1452	fk_flag_reply
C8	Moderate the same flag twice	1062	uq_mod_flag
C9	Use action "ban"	3819	ck_mod_action (only the second insert fails)
C10	Delete a category in use	1451	fk_post_category
C11	Post in category 9999	1452	fk_post_category

If any of these succeeds, a rule isn't being enforced. A C2, C3, C4, or C9 that succeeds almost always means MySQL is older than 8.0.16. Check `SELECT VERSION();`.

Part D: delete behavior (run last)

- **D1** tries to hard-delete r1. It should fail with 1451. The cascade would reach flag f1, and f1 has a moderation record. This is the audit log protecting itself.
- **D2** builds a small thread with a vote and a flag but no moderation, deletes the parent reply, and checks that everything under it is gone. All three counts should be 0.

Starting over

Re-run the schema script, then the test script. If you get error 1062 in Part A on `app_user`, you skipped the schema re-run and the sample users already exist.

The drift check

Keep Part B's query 9 handy beyond testing. Run it any time the forum pages look off. Any row it returns is a post whose stored counter disagrees with the real count, which means some code path wrote to the tables without following section 8. To repair a post's counters:

```
UPDATE post p
   SET reply_count = (SELECT COUNT(*) FROM reply r WHERE r.post_id = p.id),
       vote_score  = (SELECT COUNT(*) FROM post_vote v WHERE v.post_id =
p.id)
 WHERE p.id = ?;
```

Then find the code path that broke it. The repair only treats the symptom.

Testing from Java

The Quality & Integration Team will run tests against our container, not Workbench. The same cases make good JUnit tests: run Part A as a fixture, assert the Part B results, and assert each Part C case throws a `SQLException` with the expected `getErrorCode()`. Then test the same cases through the API endpoints, where they should come back as the HTTP codes in section 8.

11. Troubleshooting

Most problems with this schema come from three causes: running Oracle SQL in MySQL, an old MySQL version, or running statements one at a time when they need to run together.

Errors you'll actually see

Error	What it means	Fix
1064 syntax error near `FOR t IN` or `VARCHAR2`	You're running the old Oracle script	Use `subsystem5_forum_mysql.sql`
1046 No database selected	The script ran without `USE mason_forum`	Run the whole schema script, or run `USE mason_forum;` first

Error	What it means	Fix
1146 Table doesn't exist	Schema not built, or wrong database selected	Run the schema script; check the database dropdown in Workbench
1062 Duplicate entry	A unique rule fired	Expected for double votes. In Part A, means you didn't reset first
1452 Cannot add or update a child row	A foreign key target is missing or in a different thread	Check the IDs. With composite keys, check the post id too
1451 Cannot delete or update a parent row	Something still points at this row	Soft delete instead (section 5)
3819 Check constraint is violated	A value broke a check	The message names the check; look it up in section 5
3823 Column cannot be used in a check constraint	A check on a column with ON DELETE CASCADE	Only happens if someone edits the schema. Move the rule to Java
1191 Can't find FULLTEXT index	MATCH columns don't match the index	Use `MATCH(title, body)`, both columns, in that order
1175 Safe update mode	Workbench blocks UPDATE/DELETE without a key in WHERE	Use the primary key in WHERE, or run `SET SQL_SAFE_UPDATES = 0;` for that session
Variables are NULL in Part C	You ran Part C in a new tab or after reconnecting	Re-run Part A in the same tab first

Silent problems (no error at all)

These are worse than errors because nothing tells you.

- **Checks don't fire.** MySQL older than 8.0.16 accepts and ignores them. Part C's C2 will succeed. Upgrade.
- **Foreign keys don't fire.** Either the table isn't InnoDB, or someone wrote the key inline (`author_id CHAR(36) REFERENCES app_user(id)`). MySQL parses that and throws it away. Always use the table-level `CONSTRAINT ... FOREIGN KEY` form.
- **Counters are wrong.** Something wrote to the tables without the section 8 transaction. Run the drift check.

- **Search returns nothing.** The search words are under 3 characters, or all stopwords. Try longer terms.

Checking a slow query

Put `EXPLAIN` in front of any `SELECT`:

```
EXPLAIN SELECT * FROM post
 WHERE category_id = 1 AND status = 'visible'
 ORDER BY last_activity_at DESC LIMIT 20;
```

Look at the `key` column. It should name one of our indexes (here, `ix_post_category_activity`). If it says `NULL` and `type` says `ALL`, MySQL is reading every row. With our demo data that's harmless; with real data, add an index that matches the query.

Oracle to MySQL: what changed

For anyone who read the Oracle version or the earlier guide:

Oracle	MySQL	Notes
`BEGIN ... EXECUTE IMMEDIATE` drop loop	`SET FOREIGN_KEY_CHECKS = 0; DROP TABLE IF EXISTS`	MySQL has no anonymous code blocks
`VARCHAR2(n)`	`VARCHAR(n)`	Same meaning
`CLOB`	`TEXT`	64 KB limit, plenty for forum posts
`NUMBER(1)`	`TINYINT(1)`	Used as true/false
`NUMBER` / `NUMBER(10)`	`INT`	
`GENERATED BY DEFAULT AS IDENTITY`	`AUTO_INCREMENT`	
`SYS_GUID()` + REGEXP formatting	`DEFAULT (UUID())`	MySQL's UUID is already formatted
`TIMESTAMP DEFAULT SYSTIMESTAMP`	`DATETIME DEFAULT CURRENT_TIMESTAMP`	DATETIME avoids TIMESTAMP's year-2038 limit

Oracle	MySQL	Notes
Inline `REFERENCES`	Table-level `FOREIGN KEY`	MySQL ignores the inline form
`SYSTIMESTAMP - INTERVAL '7' DAY`	`NOW() - INTERVAL 7 DAY`	
Oracle Text (needs CTXAPP role)	`FULLTEXT` index	No special permissions
`ck_reply_not_own_parent`	Removed	Moved to Java (error 3823)
Explicit FK indexes required	Created automatically	Ours are kept because they're composite

12. Integration with other subsystems

In the integrated build, our forum tables sit next to Shared Core's tables, and one of them goes away: `app_user`. Everything else in this database is ours to own.

Swapping app_user for Shared Core's user table

Our `app_user` is a stand-in. The SOW says Shared Core owns the canonical User model, and no subsystem may redefine it. Before integration:

1. **Confirm the database engine.** Shared Core picks it, and the SOW recommends PostgreSQL. Foreign keys can't cross engines. If Shared Core goes with PostgreSQL, this schema needs a port, not a tweak. Ask them now, not in Week 13.
2. **Get their table and column names.** If it's called `users` with an `id` column, our 8 foreign keys that point at `app_user(id)` need to point there instead.
3. **Match the ID type exactly.** If their id is `BINARY(16)` or `BIGINT`, every `CHAR(36)` user column in our tables changes to match. Foreign keys fail when types differ.
4. **Delete our `CREATE TABLE app_user`** and its drop line, and load Shared Core's schema first.
5. **Stop reading `app_user.name` directly** if Shared Core wants other subsystems to go through `GET /users/{id}`. Ask which they prefer. Joining their table is faster; calling their API is looser coupling.

The roles in our check (`STUDENT` , `MENTOR` , `STAFF` , `ADMIN`) are our guess at Shared Core's names. Their spelling wins.

Who calls what

Other subsystem	Direction	What crosses
Shared Core	We call them	Token validation and role checks on every request; user names and roles
Notifications	We call them	`POST /notifications/dispatch` for reply received, moderation action taken, new queue item
Platform Admin & Analytics	They call us	Moderation queue items and actions for their unified dashboard; post and reply counts for analytics
Platform Admin & Analytics	They configure us	Category names and descriptions (the SOW lists this as their platform config)

What Admin & Analytics will want from these tables

Their dashboard needs aggregate counts, never individual tracking. These queries answer the SOW's forum metrics:

```
-- posts and replies per week
SELECT YEARWEEK(created_at) AS week, COUNT(*) AS posts
  FROM post GROUP BY week ORDER BY week;

-- reply rate: share of posts with at least one reply
SELECT AVG(reply_count > 0) AS reply_rate FROM post WHERE status = 'visible';

-- moderation activity
SELECT action, COUNT(*) FROM moderation_action GROUP BY action;
```

Settle with them whether they query our tables or call an endpoint we expose. Either way, it needs an entry in the project wiki's interface contracts (SOW section 6).

Before the Week 13 freeze

- [ ] Shared Core's engine and user table confirmed, and our foreign keys pointed at it

- [ ] Role names match Shared Core's spelling
- [ ] Schema script loads cleanly inside our Docker container
- [ ] Test script Parts A to D pass against the containerized database
- [ ] Notification template keys agreed with the Notifications team
- [ ] Category config ownership agreed with Admin & Analytics
- [ ] Every endpoint in section 9 documented in the project wiki
- [ ] AI prompt log entry committed to `/docs/ai-log/` for the schema and this port